

88. Greedy aur Lazy Quantifiers

Quantifiers pehli nazar me simple lagte hain par tricky ho sakte hain. Agar `(/\d+/)` se zyada complex kuch dhundhna hai to samajhna zaruri hai ki search kaise kaam karti hai.

Task

Text me saare quotes `"..."` ko guillemets `«...»` se badalna. Pehle quoted strings dhundhne hain. `(/".+"/g)` (quote, kuch, quote) sahi lagta hai, par nahi hai:

```
js
```

```
let str = 'a "witch" and her "broom" is one';  
str.match(/".+"/g);    // "witch" and her "broom"
```

2 matches (`"witch"`, `"broom"`) ki jagah **ek** bada match mila. "**Greediness saari buraiyon ki jad hai.**"

Greedy search (default)

Engine ka algorithm: string ki har position par pattern match karne ki koshish, na ho to agli position.

`"."` ke liye:

1. Pehla character `"`: engine position 3 par pehla quote dhundh leta hai.
2. Baaki pattern `"."`: dot ("newline ke alawa koi bhi") se `w` match.
3. `."` ki wajah se dot repeat hota hai aur engine ek ek character jodta jaata hai, **jab tak string ka end na aa jaye** (saare characters dot se match hote hain).
4. Ab agla pattern character `"` chahiye par string khatam. Engine samajhta hai ki `."` ne bahut zyada le liya aur **backtrack** karta hai: match ko ek character chhota karta hai.
5. Phir `"` se milaan karta hai... nahi mila, aur ek character piche...
6. Ye tab tak jab tak baaki pattern (`"`) match na ho jaye.
7. Match poora: `"witch" and her "broom"`.

Greedy mode me quantified character utni baar repeat hota hai jitni zyada se zyada possible. Engine `."` ke liye jitne characters mil sakte hain jod leta hai, phir baaki pattern match na ho to ek ek karke ghatata hai.

Lazy mode

Greedy ka ulta: "**kam se kam baar** repeat karo". Quantifier ke baad `?` lagao: `(*)?`, `(+)?`, `(?){2}`.

`?` normally khud ek quantifier hai (zero ya ek), par kisi **dusre quantifier ke baad** lagne par uska matlab greedy se lazy mode ho jaata hai.

```
js
```

```
let regexp = /\.+?/g;  
'a "witch" and her "broom" is one'.match(regexp); // "witch", "broom"
```

Search kaise chalti hai: pehle 3 steps same. Fir engine dot ko aur repeat karne ke bajay **turant** baaki pattern `()` match karke dekhta hai. Nahi mila to dot ki repetition badhata hai, aur ye tab tak jab tak baaki pattern match na ho. Agli search current match ke end se shuru.

Laziness sirf `()` wale quantifier par hoti hai, baaki greedy hi rehte hain:

```
js
```

```
"123 456".match(/\d+ \d+?/); // 123 4
```

`\d+` greedy hai to `(123)` le leta hai. Space match. `\d+?` lazy hai to sirf ek digit `(4)`. Aage kuch pattern nahi hai, isliye wahin ruk jaata hai. Match `(123 4)`.

Modern engines internally optimize karte hain to hu-ba-hu is tarah nahi bhi chal sakte, par samajhne ke liye ye kaafi hai.

Alternative approach

Regexps me aksar ek kaam ke kai tarike hote hain. Quoted strings bina lazy mode ke: `["^"]+`:

```
js
```

```
'a "witch" and her "broom" is one'.match(/"[^"]+"/g); // "witch", "broom"
```

Ye ek quote, phir 1 ya zyada **non-quotes** `[^"]`, phir closing quote dhundhta hai. `[^"]+` closing quote milte hi ruk jaata hai.

Ye lazy quantifiers ka replacement nahi, alag tarika hai. Kabhi ek chahiye, kabhi doosra.

Ek example jahan lazy fail aur ye chalta hai: `<a href="..." class="doc">` links dhundhna.

- `/<a href=".*" class="doc">/g`: ek link par chalta hai, par kai links ho to greedy `(.*)` bahut zyada le leta hai (do links ek match me).
- Lazy `/<a href=".*?" class="doc">/g`: kai links par chalta hai, par `<a href="link1" class="wrong">... <p style="" class="doc">` me galat match `(.*?)` `<a>` tag se aage `<p>` tak chala jaata hai).
- Sahi:** `/<a href="^[^"]*" class="doc">/g`. `(href)` ke andar ke saare characters sabse paas ke quote tak leta hai, bas wahi chahiye.

Summary

- **Greedy** (default): quantifier jitni zyada se zyada baar repeat kare. Baaki pattern match na ho to backtrack karke kam karta hai.
- **Lazy** (`(?)` quantifier ke baad): pehle baaki pattern match karke dekhta hai, chalta nahi to repetition badhata hai.
- Lazy mode har cheez ka ilaj nahi. Alternative: "fine-tuned greedy" with exclusions, jaise `"[^\"]+"`.

Tasks

- `"123 456".match(/\d+? \d+?/g)` -> `123 4` (pehla lazy `\d+?` space tak pahunchne ke liye `123` lena padta hai, doosra sirf ek digit).
- **HTML comments dhundhna:** `/<!--.*?-->/gs` (lazy dot `-->` se pehle ruk jaata hai; `s` flag taaki dot newline bhi match kare, multiline comments ke liye).
- **HTML tags (opening/closing, attributes ke saath):** `/<[^<>]+>/g` (assume ki attributes me `<` `>` nahi hote).